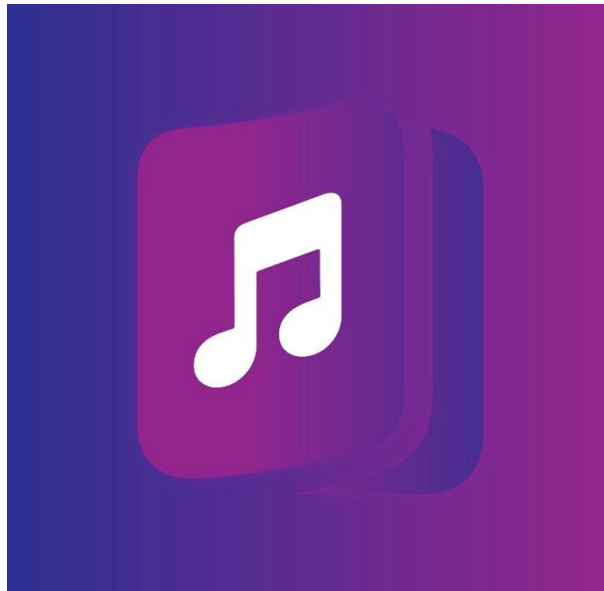


Televisió Interactiva

Cas pràctic iTunes: Fitxa tècnica



Integrants: Àlex Alonso
Ricard Colet
Marc Sánchez

Extres escollits

1. Permetre paginació

Ens demanen que l'aplicació vagi retornant els resultats per blocs. On a mesura que es va fent scroll pels resultats, al arribar al últim, s'afegeixin 20 més i així cada vegada fins que ja no hi hagin més resultats.

2. Opció de favorites

Ens demanen que la aplicació permeti que el usuari pugui tant marcar o desmarcar cançons i vídeos musicals com favorites I que aquesta informació persisteixi, tot això fent servir una base de dades natives de IOS, en el nostre cas fem servir la base de dades de UserDefaults.

3. Pantalla de preferència d'usuari

En aquest extra ens demanen de tenir en la nostre aplicació una pantalla de preferències, a on el usuari tingui diferents opcions i pugui dir localitzacions preferides, de forma que la aplicació filtri la informació segons aquelles preferències. O que també pugui indicar un altre tipus de preferències visuals.

Documentació/webs de referència

- SwiftUI Documentation – Apple Developer:
<https://developer.apple.com/documentation/SwiftUI>
- iTunes Search API – Apple Developer:
https://developer.apple.com/library/archive/documentation/AudioVideo/Conceptual/iTuneSearchAPI/index.html#//apple_ref/doc/uid/TP40017632-CH3-SW1
- Menu – Apple Developer:
<https://developer.apple.com/documentation/swiftui/menu>
- Ús de UserDefaults en IOS – Platzi:
<https://platzi.com/cursos/swiftdata-realm/userdefaults/>
- Paginació en Swift:
<https://www.youtube.com/watch?v=gKJ-yRSkt5s>
- Ús de IA com ChatGPT per solucionar diferents errors i problemes en el codi.

Passos claus de la implementació

Els passos claus alhora de la implementació del extra de la paginació son:

- Primer afegim noves propietats a **ITunesStore**, com son **allResults: [Item]** que guarda tots els resultats de l'API, després un **pageSize = 20** que es la mida dels blocs de resultats que retornem, i un **currentPage = 0** per indicar a quina pàgina actual estem per mostrar els resultats que queden.
- Després d'això el que hem fet ha sigut la funció **handleNewResults**. La qual inicialitza **allResults** i reinicia **currentPage**, per després cridar a la funció **loadNextPage** i mostrar els 20 primers resultats de l'API.
- Per últim hem implementat la funció de **loadNextPage**, funció que es crida sempre que el usuari hagi arribat mitjançant scroll a l'últim resultat, això ho fem amb un **.onAppear** posat a la view de **SearchItemsView**. Aquesta funció lo primer que fa és calcular els índexs del inici i del final de cada bloc de resultats. Per després afegir els nous resultats a l'estat **loaded** per a que d'aquesta manera el mateix SwiftUI actualitzi la llista.

Els passos claus alhora de la implementació del extra dels favorits son:

- Primer de tot hem creat la classe de **SongsFavorites**, on hi ha un array de **favorites** que emmagatzema els ítems (cançons i vídeos) preferits. On a més s'ha fet servir una base de dades, UserDefaults, i hem tingut que crear una clau que es diu **songs_favorites**.
- Durant la implementació d'aquest extra hem fet servir diferents funcions claus com es el **toggleFavorites** el qual afegeix/elimina un ítem del array. Després per comprovar si un ítem es favorit amb la funció de **isFavorite**. A continuació per guardar els resultats a la base de dades UserDefaults i codificar els favorits en JSON hem fet servir el **saveItem**, i per llegir els favorits al iniciar la app ho fem amb **init**.
- Tot això per després a les views de **ItemRowView** i **ItemDetailsView** mostrar un icona d'estrella que es podrà activar o desactivar i mirar com està.

Els passos claus alhora de la implementació del extra de la pantalla de preferències son:

- Primer de tot hem creat la view **UserView** on es mostra avatar, nom, i secció de preferències amb opcions com gènere, país i contingut explícit.
- A la vegada també implementem un **OptionsDropDownView** la qual ens ajuda per reutilitzar codi, on utilitzem **@Binding** per actualitzar a partir de l'opció escollida i un **selectAction** per poder executar la part de filtrar a partir de les opcions que ha escollit el usuari.
- Tot això per després poder connectar-ho al **ITunesStore** i poder filtrar les cerques a partir del gènere, país, i si es vol o no contingut explícit.